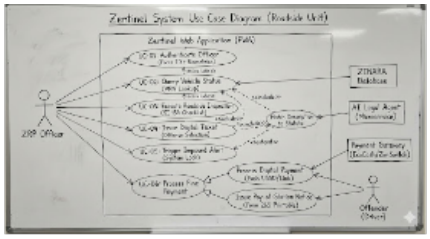
Use Case Diagram

The diagram below visualizes the boundaries of the Zentinel system. The **Primary Actor** is the ZRP Officer, while **Secondary Actors** (supporting systems) include ZINARA and the AI Legal Agent.

Primary Use Cases for Zentinel:

- **UC-01: Authenticate Officer:** Secure login via Force Number.
- **UC-02: Query Vehicle Status:** Fetching data from ZINARA using a VRN.
- **UC-03: Execute Roadside Inspection:** Filling the digital SI 154 checklist.
- **UC-04: Issue Digital Ticket:** AI calculating the fine and generating the record.
- **UC-05: Trigger Impound Alert:** The logic that flags a vehicle for seizure based on history.
- **UC-06: Process Fine Payment:** Allows the offender to pay the fine usiing digital wallets or oopt to pay cash at their nearest station within seven days.

The system will have tabs where the officer can select to either issue a quick ticket for offenses such as speeding or to choose to conduct a roadside inspection, or to query vehicle status. They do not have to do all at once but choose one at their discretion.



1. UC-01 (Authentication)

UC-01: Authenticate Officer

- **Actor:** ZRP Officer / Admin
- **Pre-conditions:** The officer has an active profile in the ZRP Personnel Database; Device has registered biometric data (if applicable).
- **Main Success Scenario:**
- Officer enters their **Force Number** and password/PIN[1].
- System validates the password hash against the encrypted Users table[2].
- **System checks for hardware biometric capability (Fingerprint/FaceID).**

- **System prompts: "Verify Identity."**
- **Officer scans fingerprint or face.**
- **System verifies the biometric signature against the device's Secure Enclave.**
- System verifies the officer's **Rank** to determine access levels (RBAC)[3].
- System generates a secure **JSON Web Token (JWT)** for the session[4].
- Officer is redirected to the **Zentinel Dashboard**[5].
- **Extensions (Exceptions):**
- **Invalid Credentials:** System logs the failed attempt and denies access[6].
- **Biometric Mismatch:** System rejects login despite correct password (prevents credential sharing).
- **Biometric Hardware Failure:** System falls back to a secondary "Manager Override PIN" or One-Time-Password (OTP) via SMS.
- **Suspended Account:** System alerts the officer to contact HQ[7].

UC-02: Query Vehicle Status

- **Actor:** ZRP Officer, ZINARA Database (Secondary)
- **Pre-conditions:** Officer is authenticated.

- **Main Success Scenario:**
- Officer enters the **Vehicle Registration Number (VRN)**.
- **Zentinel** sends a real-time API request to the **ZINARA** database.
- System retrieves: Vehicle Make/Model, Insurance Expiry, and License Disc status.
- System cross-references the internal **Zentinel History** to check for outstanding unpaid tickets.
- Results are displayed with color-coded status (Green: Compliant, Red: Non-compliant).
- **Extensions:**
- *No Connection:* System switches to **Offline Cache** to check if the vehicle was previously flagged.

UC-03: Execute Roadside Inspection

- **Actor:** ZRP Officer
- **Pre-conditions:** UC-02 is completed; vehicle is stopped.
- **Main Success Scenario:**
- Officer opens the **SI 154 Checklist**.
- Officer conducts a physical check of safety equipment (Fire Extinguisher, Reflectors, Spare Wheel, Tyres).
- Officer toggles "Pass/Fail" for each item in the Web App.
- System prompts for photo evidence if a "Fail" is recorded.
- The inspection report is temporarily saved to the session

state.

UC-04: Issue Digital Ticket

- **Actor:** ZRP Officer, AI Legal Agent (Secondary)
- **Pre-conditions:** UC-03 is completed; at least one violation or "Fail" is recorded.
- **Main Success Scenario:**
- Officer selects the specific offense.
- **AI Legal Agent** parses the **Road Traffic Act** and **SI 154**.
- AI returns the exact **Statutory Reference** and the **Deposit Fine Level**.
- Officer reviews the AI-generated ticket details.
- Officer hits "Submit."
- **Zentinel** generates an immutable UUID and sends a digital receipt to the driver's phone via SMS from the zinara system

AI Legal Assistant

- **Trigger:** Officer is unsure of the exact charge.
- **Action:** Officer types a natural description into the "Offense" field (e.g., *"No reflectors at the back"*).
- **System Logic:**
- Zentinel sends the text to the **AI Legal Agent**.
- AI performs a **Semantic Search** against the Road Traffic Act database.

- System displays the "Top 3 Most Likely Matches."
- **Action:** Officer clicks **[+] Expand** on a match to read the full legal definition.
- **Action:** Officer confirms the correct match.

UC-05: Trigger Impound Alert

- **Actor:** Zentinel System (Automated Logic)
- **Pre-conditions:** UC-03 or UC-04 are in progress.
- **Main Success Scenario:**
- The system analyzes the current "Fails" from UC-03.
- The system queries the historical database for the specific **VIN/VRN**.
- **Logic:** If Current_Safety_Status == 'Critical' OR Violation_Count > Threshold.
- The system locks the "Issue Ticket" screen and replaces it with a **Mandatory Impound Order**.
- The system notifies ZRP HQ of the impoundment for towing logistics.

UC-06: Process Fine Payment

- **Actor:** ZRP Officer, Offender (Driver), Payment Gateway (External System).
- **Pre-conditions:** A ticket has been generated, reviewed,

and confirmed. Ticket Status is currently PENDING.

- **Main Success Scenario (Digital / Instant Payment):**
- Officer presents the **Total Fine Amount** to the Offender.
- Offender agrees to pay immediately via Mobile Money (EcoCash/OneMoney) or Card.
- Officer selects **"Digital Payment"** on the tablet.
- System requests the **Payer's Mobile Number**.
- Officer inputs the number and hits **"Request Payment."**
- System sends a USSD Push / Payment Link to the Offender's device.
- Offender enters their PIN on their own device to authorize the transaction.
- System receives a **"Transaction Successful"** webhook from the Gateway.
- System updates Ticket Status to PAID.
- System generates a **Digital Receipt** (SMS/PDF).
- Officer is authorized to release the vehicle.
- **Alternative Flow A (Opt to Pay at Station / Cash):**
- *Scenario: Offender has cash but no mobile money, or refuses digital payment.*
- Offender states they wish to pay cash.
- Officer informs Offender: **"I cannot accept cash here. You must pay at the nearest Station."**

- Officer selects **"Pay at Station (Cash)"** option.
- System changes Ticket Status to AWAITING_PAYMENT.
- System generates a **"Notice to Pay" (Form 265)** with a unique Reference Number and a deadline (e.g., 7 Days).
- System syncs this debt to the **ZINARA Database** (preventing vehicle license renewal until paid).
- Officer issues the "Notice to Pay" (Print or SMS) to the Offender.
- Offender is released but remains legally liable for the debt.
- **Extensions (Exceptions):**
- **Payment Timeout:** If the driver does not enter the PIN within 60 seconds, the System prompts "Retry or Switch to Station Payment?".
- **Insufficient Funds:** Payment Gateway returns "Failed". Officer must switch to "Pay at Station" flow or impound vehicle if the offense is critical (e.g., DUI).
- **Offline Mode:** If the tablet is offline, the System cannot process Digital Payment. It defaults to **"Issue Digital Summons"** (Pay Later) automatically.

Data Flow

Level 0: The Context Diagram (The "Big Picture")

Imagine a single, large central process bubble labeled **"0.0 Zentinel System."** This bubble represents your entire web application. Surrounding it are four square boxes representing

the **External Entities**. On the left, you have the **ZRP Officer**, who sends "Login Credentials," "Vehicle Registration Numbers (VRN)," and "Inspection Inputs" into the system, and receives "Digital Tickets," "Vehicle Status Alerts," and "Impound Orders" in return. On the right, you have the **ZINARA Database**, which receives "VRN Queries" and returns "License & Insurance Status." Below is **ZRP HQ**, receiving "Revenue Reports" and "Audit Logs." Finally, you have the **AI Legal Agent**, which acts as a reference library; the system sends it an "Offense Trigger," and it returns the "Statutory Instrument Reference" and "Fine Amount."

Level 1: The Functional Decomposition (The "Main Engines")

1. **1.0 Authenticate User:** This process takes the officer's credentials and biometrics, and checks them against the **Users Data Store**.
2. **2.0 Analyze Vehicle Status:** This process takes the VRN, queries the **ZINARA API**, and simultaneously checks the internal **Vehicle History Store** to count past violations.
3. **3.0 Execute Inspection:** This handles the SI 154 Checklist inputs and saves evidence to the **Evidence Store**.
4. **4.0 Process Violation (The AI Engine):** This links the offense to the **Statutory Codes Store** to calculate the fine.
5. **5.0 Manage Impoundment:** A background process that watches the **Vehicle History Store** and overrides the ticket generation if safety thresholds are breached.
6. **6.0 Process Fine Payment:** The financial controller for the system. It receives the finalized ticket payload from the

Violation Engine and routes it based on the officer's selection (Digital vs. Station). It interfaces with **External Payment Gateways** (EcoCash/InnBucks) for immediate settlement or generates a "Notice to Pay" (Form 265) for delayed settlement. Crucially, it syncs payment status with the **ZINARA API** to place administrative locks on vehicles with unpaid fines.

Level 2: The Detailed Logic

Level 2 Breakdown for "1.0 Authenticate User"

- **1.1 Verify Credentials:** This sub-process takes the input (Force Number + Password) and hashes it. It compares the hash against the stored password in the database.
- **1.2 Validate Biometrics:** If the password matches, this step requires the officers to verify. Their identity using FaceID oor fingerprint.
- **1.3 Validate Rank Permissions:** If thecredentials matches, this step checks the "Rank" field (e.g., Constable vs. Inspector) to define what the user can see (RBAC).
- **1.4 Generate Session Token:** Upon success, this generates the JWT (JSON Web Token) used for all subsequent API calls.

Level 2 Breakdown for "2.0 Analyze Vehicle Status"

- **2.1 Request ZINARA Data:** This sends an HTTPS request to the external ZINARA endpoint. If the request times out (Offline Mode), it diverts to the **Local Cache**.
- **2.2 Aggregate Violation History:** This queries the **Tickets Data Store** for the specific VRN and counts how many "Unpaid" or "Safety" flags exist in the last 12

months.

- **2.3 Determine Status Color:** This logic merges the ZINARA result (e.g., "Expired") with the History result (e.g., "3 offenses"). It outputs a status code: Green (Clear), Yellow (Warning), or Red (Impound Risk).

Level 2 Breakdown for "3.0 Execute Inspection"

- **3.1 Render SI 154 Checklist:** This fetches the current list of required safety items (fire extinguisher, tires, etc.) from the **Statutory Store**.
- **3.2 Validate Inputs:** As the officer taps "Pass" or "Fail," this process checks if a photo is required. If "Fail" is selected, it triggers **3.3 Capture Evidence**.
- **3.3 Capture Evidence:** This activates the camera, compresses the image, and uploads it to the **Evidence Store**, linking the URL to the temporary inspection session.

Level 2 Breakdown for "4.0 Process Violation" (The AI Core)

- **4.1 Match Offense Code:** The officer selects "Worn Tires." This process looks up the corresponding unique ID in the **Statutory Store**.
- **4.2 Retrieve Legal Text (AI Lookup):** The system pulls the exact text from "Statutory Instrument 154, Section 14" to populate the ticket description automatically.
- **4.3 Calculate Final Fine:** It takes the "Base Fine" and checks for multipliers (e.g., is this a Bus?). The output is the final dollar amount.
- **4.4 Generate Unique UUID:** It creates the immutable

"Ticket ID" string and locks the record into the **Tickets Data Store**.

Level 2 Breakdown for "5.0 Manage Impoundment"

- **5.1 Check Critical Failures:** This scans the output of the inspection. If "Brakes" = Fail, it immediately flags "True."
- **5.2 Compare vs Thresholds:** It checks if Total_Violations > 3.
- **5.3 Trigger Lockout:** If either 5.1 or 5.2 is true, this process sends a "Lock" command to the Ticket Generation process, preventing a standard fine. Instead, it generates an "Impound Form 234" (fictional form number) and sends a "Tow Alert" to the **HQ Entity**.

Level 2 Breakdown for "6.0 Process Fine Payment"

- **6.1 Select Payment Channel:** This sub-process accepts the officer's input to determine the settlement method. It checks the **Device Connectivity State**. If the device is offline, it automatically disables the "Digital Payment" route and forces the "Pay at Station / Digital Summons" flow to prevent transaction errors.
- **6.2 Initiate Digital Transaction (The "Push"):** If "Digital" is selected, this process takes the **Payer's Mobile Number** and the **Fine Amount**. It constructs a secure JSON payload and sends a POST request to the **Payment Gateway Aggregator**. It then enters a "Polling State," waiting for the driver to enter their PIN on their own device.
- **6.3 Validate Payment Webhook:** This listener process waits for the encrypted callback from the Payment Gateway. It verifies the **Transaction Checksum** to ensure the payment confirmation wasn't spoofed (hacked). If verified, it changes the Ticket Status from PENDING to

PAID in the local **Transactions Store**.

- **6.4 Generate Liability Notice (Form 265):** If "Pay at Station" is selected (or transaction fails), this process generates a **Legal Debt Record**. It creates a PDF/SMS document with a unique **Reference Number** and a 7-day expiry date, which is issued to the driver instead of a receipt.
- **6.5 Sync Financial Constraints:** This final process broadcasts the status to external actors.
- **If Paid:** It notifies ZRP HQ for revenue logging.
- **If Unpaid:** It sends a "Debt Flag" to the **ZINARA Database**, logically locking the vehicle's license renewal capability until the debt is cleared.

Psuedocode for each core function

Process 1.0: Authenticate User

Objective: Securely verify the officer's identity, ensuring only active ZRP personnel can access the system. It handles encryption and role assignment.

```
BEGIN
    # Step 1: Input Validation
    IF Force_Number IS EMPTY OR Password_Input IS EMPTY THEN
        RETURN Error("Credentials Missing")
        EXIT PROCESS
```

```
    ENDIF

    # Step 2: Retrieve User Record
    READ Users_Data_Store
    SEARCH FOR User WHERE User.ForceID ==
Force_Number

    IF User NOT FOUND THEN
        RETURN Error("User Not Found")
        LOG Security_Event("Failed Login Attempt: " +
Force_Number)
        EXIT PROCESS
    ENDIF

    # Step 3: Verify Security Status
    IF User.Account_Status == "SUSPENDED" OR
User.Account_Status == "INACTIVE" THEN
        RETURN Error("Account Suspended - Contact Command
Center")
        EXIT PROCESS
    ENDIF

    # Step 4: Verify Password (Factor 1: What You Know)
    SET Stored_Hash = User.Password_Hash
    SET Input_Hash = SHA256(Password_Input + User.Salt)

    IF Input_Hash != Stored_Hash THEN
        RETURN Error("Invalid Password")
        INCREMENT User.Failed_Attempts
        EXIT PROCESS
    ENDIF

    # Step 5: Biometric Verification (Factor 2: Who You Are)
    # Note: We rely on the device's native 'Secure
Enclave' (Android Keystore/iOS Keychain)

    SET Device_Has_Biometrics =
```

```
CHECK_HARDWARE_CAPABILITY()

    IF Device_Has_Biometrics == TRUE THEN
        PROMPT "Scan Fingerprint or Face"
        RECEIVE Biometric_Result FROM
Device_Secure_Enclave

        IF Biometric_Result == "FAIL" THEN
            RETURN Error("Biometric Verification Failed. Identity
Unconfirmed.")
            LOG Security_Event("Potential Credential Sharing
Attempt: " + Force_Number)
            EXIT PROCESS
        ENDIF
    ELSE
        # Fallback for older devices or desktop admin login
        REQUIRE "2FA_SMS_Code" OR
"Manager_Override_PIN"
    ENDIF

    # Step 6: Role-Based Access Control (RBAC) Setup
    SET Session_Permissions = []
    IF User.Rank == "CONSTABLE" THEN
        SET Session_Permissions = ["ISSUE_TICKET",
"QUERY_VEHICLE"]
    ELSE IF User.Rank == "INSPECTOR" THEN
        SET Session_Permissions = ["ISSUE_TICKET",
"QUERY_VEHICLE", "VOID_TICKET", "VIEW_ANALYTICS"]
    ENDIF

    # Step 7: Session Generation
    # Only generated IF Password AND Biometrics both passed
    GENERATE JWT_Token WITH Payload (User.ForceID,
Session_Permissions, Expiry=8hrs)
    LOG Security_Event("Successful Biometric Login: " +
User.ForceID)
```

```
    RETURN JWT_Token, User.Full_Name
END
```

Process 2.0: Analyze Vehicle Status

Objective: Determine the "Risk Profile" of the vehicle by combining external ZINARA data with internal Zentinel history. This handles the "Offline" scenario.

```
PROCESS: 2.0_ANALYZE_VEHICLE_STATUS
INPUTS: VRN (Vehicle Registration Number),
GPS_Coordinates
OUTPUTS: Vehicle_Object, Risk_Level (Green/Yellow/Red),
Repeat_Offender_Count

BEGIN
    SET Vehicle_Data = NULL
    SET Is_Online = CHECK_CONNECTION()

    # Step 1: Fetch External Data (ZINARA)
    IF Is_Online == TRUE THEN
        TRY
            SEND HTTP_GET TO "ZINARA_API/vehicles/" + VRN
            RECEIVE Response_Data
            SET Vehicle_Data = Response_Data
            # Cache this data locally for future offline use
            WRITE Local_IndexedDB WITH Vehicle_Data
        CATCH Error
            SET Is_Online = FALSE # Fallback if API fails
        ENDTRY
    ENDIF

    # Fallback: Offline Mode
    IF Is_Online == FALSE THEN
        READ Local_IndexedDB
        SEARCH FOR Cached_Vehicle WHERE Plate == VRN
        IF Cached_Vehicle FOUND THEN
```

```
            SET Vehicle_Data = Cached_Vehicle
            DISPLAY Warning_Message("OFFLINE MODE: Using
cached data")
        ELSE
            RETURN Error("Vehicle not found locally. Internet
required.")
            EXIT PROCESS
        ENDIF
    ENDIF

    # Step 2: Internal History Check (The Repeat Offender
Logic)
    READ Tickets_Data_Store
    SET History_Records = SELECT * FROM Tickets WHERE
Ticket.VRN == VRN AND Ticket.Date > (TODAY - 12
MONTHS)

    SET Repeat_Offender_Count = COUNT(History_Records)
    SET Outstanding_Fines = SUM(History_Records WHERE
Payment_Status == "UNPAID")

    # Step 3: Determine Risk Status Color
    SET Status_Color = "GREEN"

    IF Vehicle_Data.License_Disk_Expiry < TODAY THEN
        SET Status_Color = "YELLOW" (Expired License)
    ENDIF

    IF Outstanding_Fines > 0 OR Repeat_Offender_Count > 3
THEN
        SET Status_Color = "RED" (High Risk)
    ENDIF

    RETURN Vehicle_Data, Status_Color,
Repeat_Offender_Count
END
```

Process 3.0: Execute Inspection (SI 154)

Objective: Manage the digital checklist, enforce data integrity, and ensure evidence is captured for failures.

```
PROCESS: 3.0_EXECUTE_INSPECTION_V2
INPUTS: VRN, List_Of_Checklist_Responses (Dictionary of Item:Pass/Fail)
OUTPUTS: Inspection_ID, List_Of_Failed_Item_IDs

BEGIN
    # Step 1: Initialize the Failure Collection
    SET List_Of_Failed_Item_IDs = []

    # Step 2: Load Statutory Requirements
    READ Statutory_Codes_Store
    SET Required_Items = GET "SI_154_Checklist_Items"

    CREATE NEW Inspection_Record
    SET Inspection_Record.VRN = VRN
    SET Inspection_Record.Timestamp = NOW()

    # Step 3: Iterate Through Responses
    FOR EACH Item IN Required_Items DO
        SET User_Response =
List_Of_Checklist_Responses[Item.ID]

        # Logic: If it fails, just remember what failed.
        IF User_Response == "FAIL" THEN
            ADD Item.ID TO List_Of_Failed_Item_IDs
            ADD (Item.ID, "FAIL") TO Inspection_Record.Details
        ELSE
            ADD (Item.ID, "PASS") TO Inspection_Record.Details
        ENDIF
    END FOR
```

```
    # Step 4: Save Record
    SET Inspection_Record.Status = "COMPLETED"
    WRITE Vehicles_Data_Store INSERT Inspection_Record

    # Step 5: Pass the list to the next process
    RETURN Inspection_Record.ID, List_Of_Failed_Item_IDs
END
```

Process 4.0: Process Violation (AI Legal Engine - V4)

Objective: To capture the offense using the officer's preferred method (Browsing vs. Typing), resolve it to a valid Legal Code, combine it with any checklist failures, and generate the final financial penalty.

```
PROCESS: 4.0_PROCESS_VIOLATION_AI_V4
INPUTS:
    Checklist_Failures_List (Optional),
    User_Selection_Mode ("DROPDOWN" or
"AI_TEXT_SEARCH"),
    User_Input_Data (Either an Integer ID or a String
Description),
    Vehicle_Class,
    Inspection_ID

OUTPUTS: Ticket_Object, Total_Fine_Amount

BEGIN
    # Step 1: Initialize the Master List of Offenses
    # Start with items found during the physical inspection (e.g.
Balding Tyres)
    SET Final_Offense_IDs = Checklist_Failures_List

    # ------------------------------------------------------
    # PHASE 1: HANDLE USER CHOICE (BRANCHING
LOGIC)
    # ------------------------------------------------------
```

```
    # Option A: The Officer chose to use the Dropdown Menu
    IF User_Selection_Mode == "DROPDOWN" THEN
        # The input is already a valid Offense_ID (e.g., 104)
        ADD User_Input_Data TO Final_Offense_IDs

    # Option B: The Officer chose to type a description
    ELSE IF User_Selection_Mode == "AI_TEXT_SEARCH"
THEN

        # 1. Clean the Officer's text (remove "the", "it", etc.)
        SET Clean_Text =
REMOVE_STOP_WORDS(User_Input_Data)

        # 2. Perform Semantic Search
        READ Statutory_Codes_Store
        SET AI_Matches = []

        FOR EACH Law IN Statutory_Codes_Store DO
            SET Score =
CALCULATE_SEMANTIC_SIMILARITY(Clean_Text,
Law.Description + Law.Keywords)
            IF Score > 0.70 THEN
                ADD (Law, Score) TO AI_Matches
            ENDIF
        END FOR

        # 3. Present Options & Require Confirmation (Interaction
Step)
        SORT AI_Matches BY Score DESCENDING

        # In a real system, the UI asks the user to pick the best
match here.
        # For logic purposes, we capture the ID of the confirmed
match.
        SET Confirmed_Match_ID =
GET_USER_CONFIRMATION(AI_Matches)
```

```
        IF Confirmed_Match_ID IS VALID THEN
            ADD Confirmed_Match_ID TO Final_Offense_IDs
        ELSE
            RETURN Error("No valid legal match found for
description.")
        ENDIF

    ENDIF

    # ----------------------------------------------------------
    # PHASE 2: CALCULATE CUMULATIVE FINE
    # ----------------------------------------------------------
    SET Total_Fine_Amount = 0
    SET Ticket_Line_Items = []

    FOR EACH Offense_ID IN Final_Offense_IDs DO

        # Look up the details for calculation
        READ Statutory_Codes_Store
        SEARCH FOR Legal_Code WHERE Code_ID ==
Offense_ID

        IF Legal_Code FOUND THEN
            SET Current_Fine = Legal_Code.Deposit_Fine_Level
            SET Display_Text = Legal_Code.Description

            # Fetch the "Expanded Explanation" (Your requested
feature)
            SET Explanation =
GET_FULL_LEGAL_TEXT(Legal_Code.Statutory_Ref)

            # Apply Multipliers (e.g., PSV Bus Surcharge)
            IF Vehicle_Class == "PSV" AND
Legal_Code.Strict_Liability == TRUE THEN
                SET Current_Fine = Current_Fine * 1.5
                APPEND " (PSV Surcharge)" TO Display_Text
```

```
            ENDIF

            SET Total_Fine_Amount = Total_Fine_Amount +
Current_Fine

            # Add to the breakdown list with the explanation
            ADD {
               "Offense": Display_Text,
               "Fine": Current_Fine,
               "Statutory_Ref": Legal_Code.Statutory_Ref,
               "Legal_Explanation": Explanation
            } TO Ticket_Line_Items
         ENDIF
      END FOR

      # ------------------------------------------------------
      # PHASE 3: GENERATE TICKET
      # ------------------------------------------------------
      CREATE NEW Ticket
      SET Ticket.UUID = GENERATE_UUID()
      SET Ticket.Inspection_Ref = Inspection_ID
      SET Ticket.Offenses_List = Ticket_Line_Items
      SET Ticket.Total_Amount = Total_Fine_Amount
      SET Ticket.Status = "ISSUED_PENDING_PAYMENT"

      WRITE Tickets_Data_Store INSERT Ticket

      RETURN Ticket
END
```

Process 5.0: Manage Impoundment (Revised)

Objective: The impound logic must scan the *list* of failures. If even *one* item in that list is a "Critical Defect," the Impound Trigger overrides the ticket.

PROCESS: 5.0_MANAGE_IMPOUNDMENT_V2

```
INPUTS: List_Of_Failed_Item_IDs, History_Count_Integer
OUTPUTS: Decision (ALLOW_TICKET or
TRIGGER_IMPOUND)

BEGIN
    SET Impound_Triggered = FALSE
    SET Impound_Reason = ""

    # Step 1: Define Critical Safety Defects (The "Kill List")
    # These IDs correspond to specific items like Brakes or
Steering
    SET Critical_Defect_IDs = ["CRIT_001_BRAKES",
"CRIT_004_STEERING",
"CRIT_009_TIRES_WIRE_EXPOSED"]

    # Step 2: Scan the current failures
    FOR EACH Failed_ID IN List_Of_Failed_Item_IDs DO
        IF Failed_ID IN Critical_Defect_IDs THEN
            SET Impound_Triggered = TRUE
            SET Impound_Reason = "Vehicle Unsafe: Critical
Defect Detected"
            BREAK LOOP
        ENDIF
    END FOR

    # Step 3: Check History (The Habitual Offender Check)
    IF Impound_Triggered == FALSE THEN
        IF History_Count_Integer >= 3 THEN
             SET Impound_Triggered = TRUE
             SET Impound_Reason = "Habitual Non-Compliance
(3+ Violations)"
        ENDIF
    ENDIF

    # Step 4: Execute Final Decision
    IF Impound_Triggered == TRUE THEN
        # Lockout Logic: Stop the ticket, start the tow truck.
```

```
        DISABLE "Issue_Fine_Button"
        DISPLAY "MANDATORY IMPOUND ORDER"
        DISPLAY Impound_Reason

        NOTIFY ZRP_HQ_Dispatch(VRN,
Reason=Impound_Reason)

        RETURN "TRIGGER_IMPOUND"
    ELSE
        # If no impound, allow the ticket from Process 4.0 to be
printed
        RETURN "ALLOW_TICKET"
    ENDIF
END
```

Process 6.0: Process Fine Payment

```
MODULE: 6.0_PROCESS_FINE_PAYMENT TRIGGER:
Ticket_Confirmed by Officer & Driver INPUTS: Ticket_Object,
Payment_Method_Select, Payer_Mobile_Number OUTPUTS:
Transaction_Receipt OR Form_265_Notice

BEGIN
    # Step 1: Environment Check (Graceful Offline)
    SET Network_Status = CHECK_CONNECTIVITY()

    # If offline, force "Pay Later" mode to prevent transaction
errors
    IF Network_Status == "OFFLINE" AND
Payment_Method_Select == "DIGITAL" THEN
        DISPLAY Alert("Offline Mode: Digital Payment
Unavailable")
        SET Payment_Method_Select = "PAY_AT_STATION"
    ENDIF

    # Step 2: Route Payment Logic
```

```
    CASE Payment_Method_Select OF

        # --- PATH A: DIGITAL PAYMENT (The "Push") ---
        WHEN "DIGITAL_PAYMENT":
            DISPLAY Screen_Input_Mobile
            INPUT Payer_Mobile_Number

            # 2.1 Call Gateway API
            SET Pay_Request = {
                "amount": Ticket_Object.Total_Fine,
                "mobile": Payer_Mobile_Number,
                "ref": Ticket_Object.UUID
            }

            SET Gateway_Response = CALL
API.Payment_Gateway.Initiate(Pay_Request)

            IF Gateway_Response.Status == "INITIATED" THEN
                DISPLAY Loading_Spinner("Waiting for Driver
PIN...")

                # 2.2 Polling / Webhook Listener (Wait for
confirmation)
                SET Payment_Verified = FALSE
                SET Timer = 0

                WHILE Timer < 60 SECONDS DO
                    SET Check_Result =
LISTEN_FOR_WEBHOOK(Ticket_Object.UUID)

                    IF Check_Result.Status == "SUCCESS" THEN
                        SET Payment_Verified = TRUE
                        BREAK LOOP
                    ENDIF

                    WAIT 2 SECONDS
                    INCREMENT Timer
```

```
            END WHILE

            # 2.3 Result Handling
            IF Payment_Verified == TRUE THEN
               UPDATE Local_DB SET Ticket.Status = "PAID"
               UPDATE Local_DB SET Ticket.Receipt_Ref =
Check_Result.Trans_ID

               SEND_SMS(Payer_Mobile_Number, "ZRP
Payment Received. Receipt: " + Check_Result.Trans_ID)
               DISPLAY Screen_Success("Payment Verified.
Safe to Release.")

            ELSE
               DISPLAY Error("Payment Timed Out or Failed")
               PROMPT "Retry OR Switch to Pay-at-Station?"
               # (Logic loops back or falls through to Path B)
            ENDIF
         ENDIF

      # --- PATH B: PAY AT STATION (Cash / Deferred) ---
      WHEN "PAY_AT_STATION":
         # 3.1 Generate Legal Debt Record
         SET Form_265_ID =
GENERATE_UNIQUE_REF("ZRP-DEBT-")
         SET Due_Date = CALCULATE_DATE(Today + 7
Days)

         UPDATE Local_DB SET Ticket.Status =
"AWAITING_PAYMENT"
         UPDATE Local_DB SET Ticket.Form_Ref =
Form_265_ID

         # 3.2 Sync Debt to ZINARA (Locks Vehicle License)
         # If offline, this ques in Service Worker to sync later
         CALL API.ZINARA.Register_Flag(Ticket_Object.VRN,
"OUTSTANDING_FINE")
```

```
        # 3.3 Issue Notice
        GENERATE_PDF Form_265_Notice(Ticket_Object,
Due_Date)
        SEND_SMS(Driver_Phone, "Fine Issued. Pay at Station
by " + Due_Date + ". Ref: " + Form_265_ID)

        DISPLAY Screen_Info("Form 265 Issued. Driver has 7
Days to pay.")

    END CASE

    # Step 4: Finalize & Close Interaction
    LOG_TRANSACTION(Officer_ID, Ticket_Object.UUID,
Timestamp)
    RETURN_TO Dashboard

END
```

Interface Design Specification

1. Navigation Architecture

The navigation remains linear to prevent officers from getting lost.

Hierarchical Breakdown:

- **Level 0: Authentication Layer**
- Login Screen (Force Number + Password)
- 2FA Prompt (Biometric)
- **Level 1: The Command Dashboard (Home)**

- **Header:** Officer Name, Station ID, Connection Status (Online/Offline).
- **Action Card A:** [Scan / Search Vehicle] (The primary entry point).
- **Action Card B:** [My History] (View tickets issued today).
- **Action Card C:** [Settings] (Dark mode, Sync Data).
- **Level 2: Operational Modules**
- Vehicle Status View (Result of search: Shows Green/Red status).
- -> Button: Start Inspection (Goes to Level 3A).
- -> Button: Quick Ticket (Goes to Level 3B).
- Impound Alert Screen (Locked screen if vehicle is critical).
- **Level 3: Input Forms**
- **3A:** SI 154 Checklist (Interactive Toggle List).
- **3B:** Offense Selector (Dual Mode)
- *Tab 1: Standard List* (Searchable dropdown of laws).
- *Tab 2: AI Assistant* (Text box for natural language description + "Explain" results).
- **Level 4: Transaction & Output**
- Review Ticket (Summary before committing).
- Success Screen (Displays generated PDF & SMS confirmation).

2. Navigation Logic (Pseudocode)

```
BEGIN
    # --- LEVEL 0: CREDENTIAL AUTHENTICATION ---
    DISPLAY Screen_Login
    INPUT Force_Number, Password

    # Check 1: Verify Password Hash
    SET Auth_Success = CALL
Auth_Service.Verify_Password(Force_Number, Password)

    IF Auth_Success == FALSE THEN
        STAY_ON Screen_Login
        DISPLAY Error_Message ("Invalid Credentials")
        EXIT MODULE
    ENDIF

    # --- LEVEL 1: BIOMETRIC VERIFICATION (New Security
Layer) ---
    # Only proceeds if password was correct
    IF Device.Has_Biometrics == TRUE THEN
        DISPLAY Prompt_Biometric ("Scan Fingerprint/Face to
Continue")
        INPUT Biometric_Scan

        SET Bio_Success = CALL
Device.SecureEnclave.Verify(Biometric_Scan)

        IF Bio_Success == FALSE THEN
            DISPLAY Error_Message ("Identity Verification Failed")
            LOG_EVENT ("Biometric Mismatch: " +
Force_Number)
            STAY_ON Screen_Login
            EXIT MODULE
        ENDIF
    ENDIF
```

```
    # --- LEVEL 2: MAIN DASHBOARD (Access Granted) ---
    GO_TO Screen_Dashboard

    WHILE ON Screen_Dashboard DO
        DISPLAY Options: [Search_Vehicle, My_History, Settings,
Logout]

        CASE User_Selection OF

            # Path A: The Core Function
            WHEN "Search_Vehicle":
                DISPLAY Screen_Vehicle_Lookup
                INPUT VRN

                SET Vehicle_Data = CALL
API.ZINARA_Lookup(VRN)

                IF Vehicle_Data.Found == TRUE THEN
                    DISPLAY Screen_Vehicle_Result (Vehicle_Data)

                    # --- BRANCH 1: FULL INSPECTION ---
                    IF User_Clicks "Start Inspection" THEN
                        GO_TO Screen_SI154_Checklist
                        # (Checklist Logic runs here)

                        # Critical Logic: Auto-Impound Check
                        IF Checklist.Result == "CRITICAL_FAIL" THEN
                            FORCE_NAVIGATE TO
Screen_Impound_Locked
                            EXIT CASE
                        ENDIF

                        GO_TO Screen_Offense_Select

                    # --- BRANCH 2: QUICK TICKET ---
                    ELSE IF User_Clicks "Quick Ticket" THEN
```

```
            GO_TO Screen_Offense_Select
        ENDIF

        # --- LEVEL 3B: OFFENSE SELECTION LOGIC ---

        WHILE ON Screen_Offense_Select DO
            DISPLAY Tabs: [Dropdown_Mode,
AI_Text_Mode]

            IF Tab == "Dropdown_Mode" THEN
                DISPLAY Searchable_List_Of_Laws
                INPUT Selection_ID

            ELSE IF Tab == "AI_Text_Mode" THEN
                DISPLAY Text_Input_Box ("Describe
Offense...")
                INPUT Officer_Description

                # Call the AI Process to get matches
                SET Matches = CALL
Process_4.0_Semantic_Search(Officer_Description)
                DISPLAY Matches_List WITH
"Expand_Explain_Button"

                INPUT User_Confirmation_ID
            ENDIF

            IF User_Clicks "Proceed_To_Review" THEN
                GO_TO Screen_Ticket_Review
                BREAK LOOP
            ENDIF
        END WHILE

        # --- LEVEL 4: TRANSACTION ---
        WHILE ON Screen_Ticket_Review DO
            DISPLAY Ticket_Summary
```

```
            IF User_Clicks "Confirm_Issue" THEN
               CALL API_Sync_Ticket
               DISPLAY Screen_Success
               WAIT 3 SECONDS
               RETURN_TO Screen_Dashboard
            ENDIF
         END WHILE

      ELSE
         DISPLAY Error "Vehicle Not Found"
         STAY_ON Screen_Vehicle_Lookup
      ENDIF

   # Path B: Officer Records
   WHEN "My_History":
      DISPLAY Screen_History_List
      IF User_Selects_Ticket THEN
         DISPLAY Screen_Ticket_Detail (Read Only)
      ENDIF
      IF User_Clicks "Back" THEN
         RETURN_TO Screen_Dashboard
      ENDIF

   # Path C: App Config
   WHEN "Settings":
      DISPLAY Screen_Settings
      ALLOW Toggle_Dark_Mode
      ALLOW Sync_Offline_Data
      IF User_Clicks "Back" THEN
         RETURN_TO Screen_Dashboard
      ENDIF

   # Path D: Exit
   WHEN "Logout":
      CLEAR Session_Token
      RETURN_TO Screen_Login
```

```
        END CASE
    END WHILE
END
```